

React Native từ cơ bản đến nâng cao

— Tập 2: Trung cấp

Quản lý state, lấy dữ liệu, lưu trữ offline, animation, device APIs, testing

Expo SDK 57 · React Native 0.86.3 · React 19.2.3 · TypeScript 6.0.3

Bản dựng: 2026-09-28 — Markdown là nguồn chính (books/react-native/vol2-trung-cap/)

Kiểm tra dấu tiếng Việt: ă ằ ẳ ẵ ơ ư ơ đ ư đ — Đường ướt, trắng khuyết, Ưu tiên

Mục lục

Chương 1 — Quản lý state: useReducer, Context, Zustand

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 2 — Lấy dữ liệu (data fetching): fetch và TanStack Query

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 3 — Lưu trữ offline: AsyncStorage và expo-sqlite

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 4 — Animation: Animated API và Reanimated

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 5 — Device APIs: rung, vị trí, chọn ảnh, quyền truy cập

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 6 — Testing: jest-expo và React Native Testing Library

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 7 — App mẫu Tập 2: "Tin Đọc Sau"

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

GLOSSARY — Thuật ngữ React Native (Anh → Việt)

Chương 1 — Quản lý state: useReducer, Context, Zustand

Mục tiêu

- Phân biệt **local state**, **global client state** và **server state**.
- Biết khi nào dùng `useState`/`useReducer`, `Context`, và một store như **Zustand**.
- Viết store Zustand có **selector** và **persist** (lưu xuống máy).
- Test store không cần render.

Giải thích đơn giản

Có 3 loại state trong một app:

Loại	Ví dụ	Công cụ trong sách
Local (cục bộ)	ô input đang gõ, modal mở/đóng	<code>useState</code> , <code>useReducer</code>
Global client state	giỏ hàng, bài yêu thích, cài đặt	Zustand
Server state	danh sách bài viết từ API	TanStack Query (Chương 2)

Ở Tập 1, app "Việc Cần Làm" dùng `useReducer` + `Context`. Cách đó ổn cho app nhỏ, nhưng có 2 nhược điểm: mọi component dùng `Context` sẽ render lại khi **bất kỳ** phần nào đổi, và phải viết `Provider`. **Zustand** là một store nhỏ: tạo bằng một hàm, dùng bằng một hook, component chỉ render lại khi **phần nó chọn** (selector) thay đổi.

Với Angular developer: Zustand store $\approx$ một `@Injectable({ providedIn: 'root' })` service có các signal bên trong; selector $\approx$ `computed()`.

Ví dụ

`useReducer` — state cục bộ, logic tập trung

`examples/src/chapters/ch01/counterReducer.ts`:

```

export type CounterAction = { type: 'inc' } | { type: 'dec' } | { type: 'reset' };

export function counterReducer(state: number, action: CounterAction): number {
  switch (action.type) {
    case 'inc':
      return state + 1;
    case 'dec':
      return Math.max(0, state - 1);
    case 'reset':
      return 0;
  }
}

```

Zustand — store toàn cục

examples/src/chapters/ch01/cartStore.ts:

```

export const useCart = create<CartState>()((set) => ({
  items: [],
  add: (item) =>
    set((s) => {
      const found = s.items.find((i) => i.id === item.id);
      return {
        items: found
          ? s.items.map((i) => (i.id === item.id ? { ...i, qty: i.qty + 1 } : i))
          : [...s.items, { ...item, qty: 1 }],
      };
    }),
  remove: (id) => set((s) => ({ items: s.items.filter((i) => i.id !== id) })),
  clear: () => set({ items: [] }),
}));

export const selectTotal = (s: Pick<CartState, 'items'>) => s.items.reduce((sum, i) =>
sum + i.price * i.qty, 0);
export const selectCount = (s: Pick<CartState, 'items'>) => s.items.reduce((n, i) => n
+ i.qty, 0);

```

Dùng trong component — không cần Provider:

```

function CartBadge() {
  const count = useCart(selectCount); // chỉ render lại khi số lượng đổi
  return <Text accessibilityLabel="Số món">🛒 {count}</Text>;
}

```

Persist — lưu store xuống máy

Store "yêu thích" của app mẫu (examples/src/state/favorites.ts) tự lưu vào AsyncStorage:

```

export const useFavorites = create<FavoritesState>()(
  persist(
    (set) => ({
      ids: [],
      toggle: (id) =>
        set((s) => ({ ids: s.ids.includes(id) ? s.ids.filter((x) => x !== id) :
[...s.ids, id] })),
      clear: () => set({ ids: [] }),
    }),
    {
      name: 'favorites-v1', // khóa trong AsyncStorage
      storage: createJSONStorage(() => AsyncStorage),
    },
  ),
);

export const useIsFavorite = (id: number) => useFavorites((s) => s.ids.includes(id));

```

Tắt app mở lại, danh sách yêu thích vẫn còn.

Test store — không cần render

Store Zustand dùng được ngoài React qua `getState()`:

```

beforeEach(() => useCart.getState().clear());

it('store Zustand: thêm cùng món thì tăng qty', () => {
  const { add } = useCart.getState();
  add({ id: 'a', name: 'A', price: 10000 });
  add({ id: 'a', name: 'A', price: 10000 });
  add({ id: 'b', name: 'B', price: 5000 });
  const state = useCart.getState();
  expect(selectCount(state)).toBe(3);
  expect(selectTotal(state)).toBe(25000);
});

```

Kết quả thật (2026-09-28):

```

PASS src/chapters/ch01/state.test.tsx
  ✓ counterReducer không cho số âm
  ✓ store Zustand: thêm cùng món thì tăng qty
  ✓ bài tập: remove và formatVnd
  ✓ nhiều component đọc cùng store và cùng cập nhật
Tests:      4 passed, 4 total
PASS src/state/favorites.test.ts
  useFavorites (Zustand + persist)
    ✓ toggle thêm rồi bỏ một id
    ✓ tự lưu xuống AsyncStorage dưới khóa favorites-v1

```

Trên Expo Go: tab **Lab** → "Ch.1 — Zustand vs useReducer" (NOT RUN trong sandbox).

Đi sâu

So sánh các lựa chọn

Thư viện	Stars (2026-09-28)	License	Hợp với ai
Zustand	58.8k	MIT	Hầu hết app; ít code
Redux Toolkit	11.2k	MIT	Team lớn, quen NgRx, cần DevTools/time-travel
Jotai	21.3k	MIT	Thích mô hình "atom" giống Signals

Chi tiết (last commit, nguồn) ở [STACK.md](#). Quyết định của sách: **Zustand** cho client state và **TanStack Query** cho server state — không trộn dữ liệu API vào store.

Selector và render lại

`useCart((s) => s.items)` trả về mảng mới mỗi khi `items` đổi → component render lại.
`useCart(selectCount)` trả về một **số**; nếu số không đổi, không render lại. Tránh selector trả về **object mới** mỗi lần `((s) => ({ a: s.a, b: s.b }))`. Hướng dẫn nâng cấp lên Zustand v5 nói rõ: selector trả tham chiếu mới "may cause infinite loops" (lỗi "Maximum update depth exceeded"). Dùng nhiều selector nhỏ, hoặc `useShallow` từ `zustand/react/shallow`.

Persist: version và migrate

Khi đổi cấu trúc state, tăng tên khóa (`favorites-v2`) hoặc dùng tùy chọn `version + migrate` của middleware persist để chuyển dữ liệu cũ. `partialize` để chỉ lưu một phần state.

Lỗi và bẫy thường gặp

- **Để dữ liệu API trong Zustand** → phải tự viết loading/error/cache/refetch. Dùng TanStack Query.
- **Selector tạo object mới** → có thể lặp vô hạn ở Zustand v5. Dùng `useShallow`.
- **State rò giữa các test** vì store là singleton → `beforeEach(() => {
 useStore.getState().clear();
})`.
- **Gọi `act()` không cần thiết** quanh `getState()` → RNTL 14 cảnh báo "You called `act(async () => ...)` without `await`". Store không cần `act` khi không có component.
- **Lưu dữ liệu nhạy cảm** (token) bằng persist + AsyncStorage → **không mã hóa**. Dùng `expo-secure-store` (Tập 3).

Tóm tắt

- Local → `useState/useReducer`; global client → Zustand; server → TanStack Query.
- Zustand: không Provider, selector để giảm render, `persist` để lưu.
- Test store bằng `getState()`; reset giữa các test.

Bài tập (có lời giải)

Bài 1. Thêm hành động `remove(id)` vào giỏ hàng và hàm `formatVnd(amount)` hiển thị 1234567 thành 1.234.567 đ.

▼ Lời giải

```
remove: (id) => set((s) => ({ items: s.items.filter((i) => i.id !== id) })),  
  
export function formatVnd(amount: number): string {  
  return `${amount.toString().replace(/\B(?=(\d{3})+(?!\d))/g, '.')} đ`;  
}
```

Test: `useCart.getState().remove('a')` → `items` rỗng; `formatVnd(1234567) === '1.234.567 đ'`. Có thể dùng `Intl.NumberFormat('vi-VN', { style: 'currency', currency: 'VND' })`, nhưng kết quả phụ thuộc dữ liệu locale của engine (Hermes trên máy thật so với Node trong test), nên sách dùng hàm tự viết để test ổn định.

Bài 2. Màn hình giỏ hàng cần hiển thị cả `count` và `total`. Viết sao cho component chỉ render lại khi một trong hai giá trị đổi.

▼ Lời giải

Cách 1 — hai selector riêng (dùng trong `CartDemo.tsx`: `CartBadge` và `CartTotal` là 2 component):

```
const count = useCart(selectCount);  
const total = useCart(selectTotal);
```

Cách 2 — một selector trả object, bọc `useShallow`:

```
import { useShallow } from 'zustand/react/shallow';  
const { count, total } = useCart(useShallow((s) => ({ count: selectCount(s), total:  
  selectTotal(s) })));
```

`useShallow` so sánh từng thuộc tính thay vì so sánh tham chiếu object. (Cách 2 là lời giải tham khảo, không có trong code dự án.)

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Zustand — Persisting store data: <https://github.com/pmndrs/zustand/blob/main/docs/reference/integrations/persisting-store-data.md>
- Zustand — Testing guide: <https://github.com/pmndrs/zustand/blob/main/docs/learn/guides/testing.md>
- Zustand — Migrating to v5 (stable selector outputs): <https://github.com/pmndrs/zustand/blob/main/docs/reference/migrations/migrating-to-v5.md>
- Zustand repo: <https://github.com/pmndrs/zustand>
- Redux Toolkit repo: <https://github.com/reduxjs/redux-toolkit>
- Jotai repo: <https://github.com/pmndrs/jotai>
- React — Scaling up with Reducer and Context: <https://github.com/reactjs/react.dev/blob/main/src/content/learn/scaling-up-with-reducer-and-context.md>

Chương 2 — Lấy dữ liệu (data fetching): fetch và TanStack Query

Mục tiêu

- Viết một "HttpClient" nhỏ trên `fetch`: `baseUrl`, `timeout`, lỗi rõ ràng.
- Dùng **TanStack Query**: `useQuery`, `query key`, `staleTime`, `retry`, `refetch`.
- Dùng `useMutation` với **optimistic update** (cập nhật lạc quan) và `rollback`.
- Kết nối TanStack Query với trạng thái mạng và vòng đời app trên React Native.
- Test bằng `fetch` giả.

Giải thích đơn giản

React Native có sẵn `fetch` giống trình duyệt. Nhưng một màn hình thật cần nhiều hơn: đang tải, lỗi, thử lại, cache, tải lại khi quay lại app... Trong Angular, bạn viết `service` + `HttpClient` + `RxJS` + tự cache. Trong React, **TanStack Query** làm hết phần đó:

- Bạn đưa **query key** (địa chỉ cache, ví dụ `['posts', 'list', 20]`) và **query function** (hàm trả Promise).
- Hook trả về `{ data, isPending, isError, error, refetch, ... }`.
- Dữ liệu được cache; màn hình khác dùng cùng key sẽ **không gọi mạng lại**.

API dùng trong sách: **JSONPlaceholder** (<https://jsonplaceholder.typicode.com>), một REST API giả miễn phí, trả `/posts` với các trường `userId`, `id`, `title`, `body`.

Ví dụ

1. Client nhỏ trên fetch

`examples/src/api/client.ts` (trích):

```

export class ApiError extends Error {
  constructor(message: string, public readonly status: number | null) {
    super(message);
    this.name = 'ApiError';
  }
}

export function createApiClient({ baseUrl, timeoutMs = 10_000, fetchImpl }:
ApiClientOptions): ApiClient {
  async function request<T>(path: string, init: RequestInit): Promise<T> {
    const doFetch = fetchImpl ?? fetch; // đọc fetch lúc gọi để test có thể thay
    global.fetch

    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(), timeoutMs);
    try {
      const res = await doFetch(`${baseUrl}${path}`, {
        ...init,
        headers: { Accept: 'application/json', 'Content-Type': 'application/json',
        ...init.headers },
        signal: controller.signal,
      });
      if (!res.ok) throw new ApiError(`HTTP ${res.status}`, res.status);
      return (await res.json()) as T;
    } catch (e) {
      if (e instanceof ApiError) throw e;
      const aborted = e instanceof Error && e.name === 'AbortError';
      throw new ApiError(aborted ? 'Hết thời gian chờ (timeout)' : 'Không kết nối được
máy chủ', null);
    } finally {
      clearTimeout(timer);
    }
  }
  // ...
}

```

- `fetch` **không** ném lỗi khi HTTP 404/500 — phải tự kiểm tra `res.ok` (khác `HttpClient`).
- `fetch` không có timeout mặc định → dùng `AbortController`.
- Tham số `fetchImpl = "inject"` một `fetch` giả khi test (giống DI).

Query key và các hàm gọi API (`src/api/posts.ts`):

```

export const postKeys = {
  all: ['posts'] as const,
  list: (limit: number) => ['posts', 'list', limit] as const,
  detail: (id: number) => ['posts', 'detail', id] as const,
};

export function fetchPosts(limit = 20, client: ApiClient = api): Promise<Post[]> {
  return client.getJson<Post[]>(`/posts?_limit=${limit}`);
}

```

2. useQuery

examples/src/chapters/ch02/PostsDemo.tsx:

```
export function PostsDemo({ limit = 5 }: { limit?: number }) {
  const query = useQuery({ queryKey: postKeys.list(limit), queryFn: () => fetchPosts(limit)
});

if (query.isPending) return <ActivityIndicator accessibilityLabel="Đang tải" />;
if (query.isError)
  return (
    <View style={{ padding: 16, gap: 8 }}>
      <Text>Lỗi: {query.error.message}</Text>
      <Pressable accessibilityRole="button" onPress={() => query.refetch()}>
        <Text>Thử lại</Text>
      </Pressable>
    </View>
  );
return (
  <View style={{ padding: 16, gap: 8 }}>
    {query.data.map((p) => <Text key={p.id}>• {p.title}</Text>)}
  </View>
);
}
```

Sau khi kiểm tra `isPending` và `isError`, TypeScript biết chắc `query.data` có giá trị.

Layout gốc tạo **một** `QueryClient` cho cả app (`src/app/_layout.tsx`):

```
const [queryClient] = useState(createQueryClient); // tạo đúng một lần
useEffect(() => setupReactNativeManagers(), []);
return <QueryClientProvider client={queryClient}>{/* Stack */}</QueryClientProvider>;
```

3. Test với fetch giả

```
const fetchMock = jest.fn();
beforeEach(() => {
  fetchMock.mockReset();
  globalThis.fetch = fetchMock as unknown as typeof fetch;
});

it('lỗi → nút Thử lại gọi lại API', async () => {
  fetchMock.mockResolvedValueOnce(jsonResponse({}),
500)).mockResolvedValueOnce(jsonResponse(POSTS));
  const user = userEvent.setup();
  await renderWithProviders(<PostsDemo limit={3} />);
  expect(await screen.findByText('Lỗi: HTTP 500')).toBeOnTheScreen();
  await user.press(screen.getByRole('button', { name: 'Thử lại' }));
  expect(await screen.findByText('• Học React Native trong 30 ngày')).toBeOnTheScreen();
});
```

`renderWithProviders` (`src/test/renderWithProviders.tsx`) tạo một `QueryClient` mới cho mỗi test với `retry: false` — đúng khuyến nghị trong hướng dẫn Testing của TanStack Query.

Kết quả (2026-09-28):

```
PASS src/chapters/ch02/fetching.test.tsx
  ✓ hiện danh sách sau khi tải
  ✓ lỗi → nút Thử lại gọi lại API
  ✓ bài tập: optimistic update rồi rollback khi server lỗi
Tests: 3 passed, 3 total
PASS src/api/client.test.ts
  createApiClient
    ✓ ghép baseUrl và trả JSON
    ✓ HTTP 404 → ApiError có status
    ✓ lỗi mạng → ApiError status null
    ✓ quá thời gian → hủy request (AbortController)
```

Đi sâu

Mặc định quan trọng của TanStack Query

Theo trang "Important Defaults":

- Dữ liệu trong cache mặc định bị coi là **stale** (cũ) ngay → dễ refetch. Đặt `staleTime` để giảm gọi mạng.
- Query lỗi được **thử lại 3 lần** với độ trễ tăng dần (exponential backoff) trước khi báo lỗi.
- Query không còn ai dùng bị xóa khỏi cache sau **5 phút** (`gcTime`).

App mẫu đặt `retry: 2`, `staleTime: 30_000` (toàn cục) và `staleTime: 60_000` cho danh sách bài. Vì có `retry`, test "mất mạng và chưa có cache" của app phải chờ lâu hơn (`findByText(..., {}, { timeout: 10_000 })`).

React Native: mạng và foreground

Trên web, TanStack Query tự refetch khi có mạng lại và khi tab được focus. Trên RN, bạn phải nối tay — theo trang "React Native" của TanStack Query. App mẫu làm trong `src/lib/queryClient.ts`:

```

onlineManager.setEventListener((setOnline) => {
  let initialised = false;
  const sub = Network.addNetworkStateListener((state) => {
    initialised = true;
    setOnline(!state.isConnected);
  });
  Network.getNetworkStateAsync().then((state) => { if (!initialised)
setOnline(!state.isConnected); }).catch(() => {});
  return () => sub.remove();
});

const onAppStateChange = (status: AppStateStatus) => {
  if (Platform.OS !== 'web') focusManager.setFocused(status === 'active');
};
AppState.addListener('change', onAppStateChange);

```

`expo-network` có trong Expo Go SDK 57, không cần cài thêm native code.

So với Angular

Angular	TanStack Query
<code>http.get<Post[]>(url)</code> trả Observable	<code>queryFn: () => fetchPosts()</code> trả Promise
<code>shareReplay(1)</code> để cache	cache theo <code>queryKey</code> , tự động
<code>retry(3)</code>	<code>retry: 3</code> (mặc định)
<code>async pipe + `*ngIf="data\$"</code>	<code>async as data"</code>
Interceptor thêm header	hàm <code>request()</code> trong client

Lỗi và bẫy thường gặp

- **Quên kiểm tra `res.ok`** → xử lý trạng lỗi HTML như JSON.
- **Tạo `queryClient` trong thân component** mà không có `useState` → mỗi lần render một cache mới.
- **Query key không chứa tham số** (`['posts']` cho mọi `limit`) → hiển thị nhầm dữ liệu.
- **Test dùng chung `QueryClient`** → test sau thấy cache của test trước.
- **Client "bắt" `fetch` lúc tạo** (`const f = fetch` ở cấp module) → test thay `globalThis.fetch` sau đó sẽ không có tác dụng. Vì vậy client đọc `fetch` **lúc gọi** (`fetchImpl ?? fetch` trong `request`).
- **HTTP (không S) trên iOS** bị App Transport Security chặn. Luôn dùng HTTPS.

Tóm tắt

- `fetch` + client nhỏ: `baseUrl`, `timeout`, lỗi rõ ràng, inject được.
- TanStack Query quản cache/loading/error/retry; key là địa chỉ cache.
- Trên RN, nối `onlineManager` với `expo-network` và `focusManager` với `AppState`.

Bài tập (có lời giải)

Bài 1. Viết hook `useCreatePost(limit)` dùng `useMutation`. Khi gọi `mutate`, bài mới phải hiện ngay đầu danh sách; nếu server lỗi, danh sách trở lại như cũ. Viết test.

▼ Lời giải

`examples/src/chapters/ch02/useCreatePost.ts`:

```
export function useCreatePost(limit: number) {
  const qc = useQueryClient();
  const key = postKeys.list(limit);

  return useMutation({
    mutationFn: (input: Pick<Post, 'title' | 'body' | 'userId'>) =>
      createPost(input),
    onMutate: async (input) => {
      await qc.cancelQueries({ queryKey: key });
      const previous = qc.getQueryData<Post[]>(key);
      const optimistic: Post = { ...input, id: -Date.now() }; // id âm = tạm thời
      qc.setQueryData<Post[]>(key, (old = []) => [optimistic, ...old]);
      return { previous };
    },
    onError: (_err, _input, context) => {
      if (context?.previous) qc.setQueryData(key, context.previous);
    },
    onSettled: () => qc.invalidateQueries({ queryKey: key }),
  });
}
```

Test giữ Promise của `fetch` ở trạng thái chờ, kiểm tra bài mới đã ở đầu danh sách, rồi mới `reject` để kiểm tra rollback (xem "bài tập: optimistic update rồi rollback khi server lỗi"). Lưu ý: JSONPlaceholder **không lưu thật** bài mới (API giả), nên sau `invalidateQueries` danh sách trên máy thật sẽ trở về dữ liệu gốc.

Bài 2. Màn hình chi tiết `/post/[id]` không nên gọi mạng nếu bài đã có trong danh sách. Làm thế nào?

▼ Lời giải

Dùng `initialData` lấy từ cache danh sách (`src/app/post/[id].tsx`):

```
const list = usePosts();
const fromList = list.data?.posts.find((p) => p.id === postId);
const { data: post } = useQuery({
  queryKey: postKeys.detail(postId),
  queryFn: () => fetchPost(postId),
  initialData: fromList,
  enabled: Number.isFinite(postId),
});
```

Test tích hợp "tải bài viết từ mạng, mở chi tiết" trong `src/__tests__/app.test.tsx` mở màn hình chi tiết rồi kiểm tra `expect(fetchMock).toHaveBeenCalledTimes(1)` — chỉ có một lần gọi mạng (cho danh sách); chi tiết dùng `initialData`.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- TanStack Query — React Native: <https://github.com/TanStack/query/blob/main/docs/framework/react/react-native.md>
- TanStack Query — Important Defaults: <https://github.com/TanStack/query/blob/main/docs/framework/react/guides/important-defaults.md>
- TanStack Query — Query Keys: <https://github.com/TanStack/query/blob/main/docs/framework/react/guides/query-keys.md>
- TanStack Query — Mutations: <https://github.com/TanStack/query/blob/main/docs/framework/react/guides/mutations.md>
- TanStack Query — Testing: <https://github.com/TanStack/query/blob/main/docs/framework/react/guides/testing.md>
- React Native — Networking: <https://github.com/facebook/react-native-website/blob/main/docs/network.md>
- Expo — expo-network (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/network.mdx>
- JSONPlaceholder (README và data.json): <https://github.com/typicode/jsonplaceholder>

Chương 3 — Lưu trữ offline: AsyncStorage và expo-sqlite

Mục tiêu

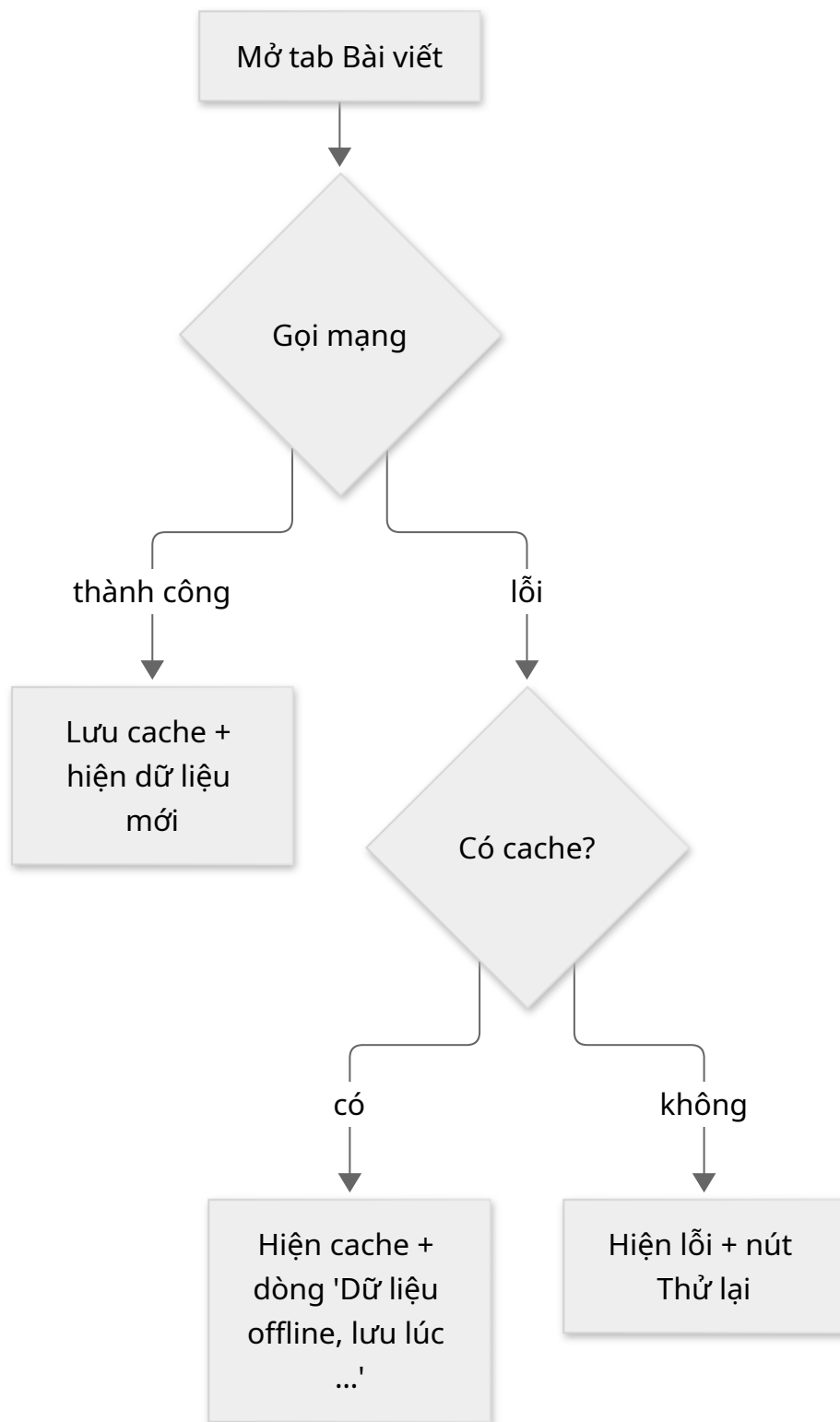
- Lưu dữ liệu xuống máy để app dùng được khi **mất mạng**.
- Dùng **AsyncStorage** (key-value) và **expo-sqlite** (cơ sở dữ liệu SQL).
- Thiết kế một **repository** có 2 cách cài đặt, chọn được khi chạy — và test được.
- Áp dụng chiến lược **network-first, cache fallback** cho app mẫu.

Giải thích đơn giản

	AsyncStorage	expo-sqlite
Mô hình	key → chuỗi (giống <code>localStorage</code> , nhưng bất đồng bộ)	bảng, SQL
Hợp với	cài đặt, token không nhảy cảm, cache nhỏ dạng JSON	nhiều bản ghi, tìm kiếm, sắp xếp, quan hệ
Mã hóa	Không	Không (SQLCipher có nhưng không dùng được trong Expo Go)
Trong Expo Go SDK 57	Có (<code>@react-native-async-storage/async-storage@2.2.0</code>)	Có (<code>expo-sqlite@57.0.3</code>)

Dữ liệu nhạy cảm (mật khẩu, token) → **expo-secure-store** (Tập 3).

Chiến lược của app mẫu:



Ví dụ

Một interface, hai cách cài đặt

`examples/src/storage/postsCache.ts` :

```

export interface CachedPosts {
  posts: Post[];
  savedAt: number; // epoch ms
}

export interface PostsCache {
  load(): Promise<CachedPosts | null>;
  save(posts: Post[], now?: number): Promise<void>;
  clear(): Promise<void>;
}

const KEY = 'posts-cache-v1';

export const asyncStoragePostsCache: PostsCache = {
  async load() {
    const raw = await AsyncStorage.getItem(KEY);
    if (!raw) return null;
    try {
      return JSON.parse(raw) as CachedPosts;
    } catch {
      return null; // dữ liệu hỏng → coi như chưa có cache
    }
  },
  async save(posts, now = Date.now()) {
    await AsyncStorage.setItem(KEY, JSON.stringify({ posts, savedAt: now } satisfies
CachedPosts));
  },
  async clear() {
    await AsyncStorage.removeItem(KEY);
  },
};

```

Bản SQLite (`src/storage/sqlitePostsCache.ts`, trích):

```

export async function migrate(db: SqlDb): Promise<void> {
  await db.execAsync(`
    CREATE TABLE IF NOT EXISTS posts (
      id INTEGER PRIMARY KEY NOT NULL,
      user_id INTEGER NOT NULL,
      title TEXT NOT NULL,
      body TEXT NOT NULL
    );
    CREATE TABLE IF NOT EXISTS meta (key TEXT PRIMARY KEY NOT NULL, value TEXT NOT
NULL);
  `);
}

// save(): xóa cũ, chèn mới trong một transaction
await db.execAsync('BEGIN');
try {
  await db.runAsync('DELETE FROM posts');
  for (const p of posts) {
    await db.runAsync('INSERT INTO posts (id, user_id, title, body) VALUES (?, ?, ?,
?)', p.id, p.userId, p.title, p.body);
  }
  await db.runAsync('INSERT OR REPLACE INTO meta (key, value) VALUES (?, ?)',
'savedAt', String(now));
  await db.execAsync('COMMIT');
} catch (e) {
  await db.execAsync('ROLLBACK');
  throw e;
}

// Tìm kiếm ngay trong SQLite (điều AsyncStorage không làm được).
export function searchCachedPosts(db: SqlDb, keyword: string) {
  return db.getAllAsync<PostRow>('SELECT id, user_id, title, body FROM posts WHERE
title LIKE ? ORDER BY id', `%${keyword}%`);
}

```

`SqlDb` là interface nhỏ gồm `execAsync`, `runAsync`, `getAllAsync`, `getFirstAsync` — đúng tên các hàm của `SQLiteDatabase` trong expo-sqlite SDK 57. Nhờ vậy:

- Trên iPhone: truyền database thật từ `SQLite.openDatabaseAsync('book-demo.db')`.
- Trong test: truyền một adapter bọc `node:sqlite` (có sẵn trong Node 22) — **SQL chạy thật**, không phải mock (`src/test/nodeSqliteDb.ts`).

Network-first, cache fallback

`examples/src/features/posts/loadPosts.ts`:

```

export async function loadPosts(
  deps: { fetcher?: () => Promise<Post[]>; cache?: PostsCache; now?: () => number } =
    {},
): Promise<PostsResult> {
  const { fetcher = () => fetchPosts(20), cache = asyncStoragePostsCache, now =
    Date.now } = deps;
  try {
    const posts = await fetcher();
    const savedAt = now();
    await cache.save(posts, savedAt);
    return { posts, source: 'network', savedAt };
  } catch (error) {
    const cached = await cache.load();
    if (cached) return { ...cached, source: 'cache' };
    throw error;
  }
}

```

Hàm này là `queryFn` của `usePosts()` (Chương 2). Màn hình chỉ cần xem `data.source`.

Test — kết quả thật (2026-09-28)

```

PASS src/storage/postsCache.test.ts
  ✓ isFresh theo TTL
  asyncStoragePostsCache
    ✓ lưu và đọc lại cùng thời điểm lưu
    ✓ dữ liệu hỏng → null thay vì crash
  createSqlitePostsCache (chạy SQL thật bằng node:sqlite)
    ✓ migrate → save → load → search
    ✓ save lần 2 thay thế toàn bộ dữ liệu cũ
PASS src/features/posts/loadPosts.test.ts
  loadPosts (network-first, cache fallback)
    ✓ có mạng: trả dữ liệu mạng và lưu cache
    ✓ mất mạng: dùng cache
    ✓ mất mạng và chưa có cache: ném lỗi gốc
PASS src/chapters/ch03/offline.test.tsx
  ✓ bài tập: decideCache theo TTL
  ✓ SqliteDemo: mở DB, lưu dữ liệu, tìm theo từ khóa
  ✓ adapter node:sqlite dùng được độc lập

```

(Node in thêm dòng "ExperimentalWarning: SQLite is an experimental feature" — đó là cảnh báo của `node:sqlite` trong Node 22, không phải lỗi.)

Component `SqliteDemo` (tab **Lab** → "Ch.3") mở SQLite thật trên máy. Trong test, ta thay `expo-sqlite` bằng `adapter`:

```

jest.mock('expo-sqlite', () => ({
  openDatabaseAsync: async () => require('@test/nodeSqliteDb').createNodeSqliteDb(),
}));

```

Chạy trên Expo Go: NOT RUN.

Đi sâu

AsyncStorage trong test

Thư viện cung cấp sẵn bản giả lưu trong bộ nhớ. `jest.setup.js` của dự án:

```
jest.mock('@react-native-async-storage/async-storage', () =>
  require('@react-native-async-storage/async-storage/jest/async-storage-mock'),
);
```

expo-sqlite: vài điểm theo tài liệu SDK 57

- `openDatabaseAsync(name)` mở (hoặc tạo) file database.
- `execAsync` chạy nhiều câu lệnh, **không escape tham số** → chỉ dùng cho SQL cố định (tạo bảng).
- `runAsync(sql, ...params)` cho ghi; `getAllAsync/getFirstAsync` cho đọc; luôn dùng `?`.
- Có `SQLiteProvider` + `useSQLiteContext()` để chia sẻ một kết nối cho cả cây component.
- Web: cần cấu hình Metro cho file `.wasm` và header COEP/COOP (dự án có `metro.config.js` cho việc này, vì script kiểm tra chạy `expo export --platform web`). Web support của expo-sqlite đang **alpha**.

Migration

Khi đổi cấu trúc bảng ở phiên bản sau, lưu số phiên bản (ví dụ `PRAGMA user_version`) và chạy các bước nâng cấp tuần tự. Tài liệu expo-sqlite có ví dụ `migrateDbIfNeeded` dùng với `SQLiteProvider` `onInit`.

Chọn cái nào?

- Dưới vài trăm bản ghi, chỉ đọc cả khối → `AsyncStorage` (đơn giản).
- Cần tìm kiếm/lọc/sắp xếp/phân trang → `SQLite`.
- App mẫu dùng `AsyncStorage` cho cache danh sách (20 bài) và `Zustand persist`; `SQLite` được dạy qua `SqliteDemo`.

Lỗi và bẫy thường gặp

- **Nối chuỗi SQL với dữ liệu người dùng** → SQL injection. Luôn dùng tham số `?`.
- `JSON.parse` **dữ liệu hỏng** làm crash lúc mở app → bọc `try/catch`, trả `null`.
- **Quên `await`** khi ghi → dữ liệu chưa kịp lưu khi app bị tắt.
- **Thiếu `expo-asset`**: khi chạy test của màn hình dùng expo-sqlite, Jest báo `Cannot find module 'expo-asset' from 'node_modules/expo-sqlite/build/hooks.js'`. Chúng tôi gặp lỗi này; cách sửa: `npx expo install expo-asset` (Expo tự thêm config plugin).

- **Web export lỗi** Unable to resolve module ./wa-sqlite/wa-sqlite.wasm → thêm metro.config.js như trên.
- **Lưu token trong AsyncStorage** → không mã hóa. Dùng expo-secure-store.

Tóm tắt

- AsyncStorage: key-value đơn giản; SQLite: truy vấn mạnh. Cả hai có trong Expo Go.
- Tách interface repository → đổi cách lưu không đổi màn hình, và test được.
- Network-first + cache fallback giúp app dùng được khi mất mạng.

Bài tập (có lời giải)

Bài 1. Viết `decideCache(cached, now, ttlMs)` trả về `'fetch'` (chưa có cache), `'use-cache'` (cache còn tươi) hoặc `'use-cache-and-refresh'` (cache cũ: hiện trước, tải mới ở nền).

▼ Lời giải

examples/src/chapters/ch03/cachePolicy.ts:

```
export function decideCache(cached: CachedPosts | null, now: number, ttlMs: number): CacheDecision {
  if (!cached || cached.posts.length === 0) return 'fetch';
  return isFresh(cached.savedAt, now, ttlMs) ? 'use-cache' : 'use-cache-and-refresh';
}
```

`isFresh(savedAt, now, ttl) = now - savedAt < ttl`. Đây chính là ý tưởng "stale-while-revalidate" mà TanStack Query dùng với `staleTime`.

Bài 2. Đổi app mẫu sang dùng SQLite thay cho AsyncStorage mà **không sửa** `loadPosts`.

▼ Lời giải

`loadPosts` nhận `cache` qua tham số, nên chỉ cần truyền bản SQLite:

```
const db = await SQLite.openDatabaseAsync('posts.db');
await migrate(db);
const sqliteCache = createSqlitePostsCache(db);
// trong usePosts:
queryFn: () => loadPosts({ cache: sqliteCache }),
```

Đây là "Dependency Inversion": `loadPosts` phụ thuộc interface `PostsCache`, không phụ thuộc `AsyncStorage`. Test `createSqlitePostsCache` với `node:sqlite` đã chứng minh bản SQLite thỏa interface.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Expo — AsyncStorage (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/async-storage.mdx>
- Expo — SQLite (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/sqlite.mdx>
- Expo — cấu hình Metro cho SQLite web: <https://github.com/expo/expo/blob/main/docs/public/static/diffs/sqlite-web-metro-config.diff>
- Async Storage repo: <https://github.com/react-native-async-storage/async-storage>
- Node.js — `node:sqlite` (tài liệu API trong repo Node): <https://github.com/nodejs/node/blob/main/doc/api/sqlite.md>

Chương 4 — Animation: Animated API và Reanimated

Mục tiêu

- Làm animation đơn giản với **Animated API** có sẵn trong React Native.
- Dùng **Reanimated 4** (có trong Expo Go SDK 57): `shared value`, `withSpring`, `withTiming`, `withSequence`, `layout animation` (`entering`, `exiting`, `layout`).
- Viết code tương thích **React Compiler** (luật lint mới).
- Test animation trong Jest.

Giải thích đơn giản

Animation là **thay đổi một giá trị theo thời gian** (opacity 0 → 1, scale 1 → 1.4 → 1) và gán giá trị đó vào style. Điều quan trọng trên mobile: animation nên chạy trên **UI thread** (luồng giao diện native) để mượt 60 khung hình/giây kể cả khi JavaScript đang bận.

- **Animated API** (có sẵn): `new Animated.Value(0)` + `Animated.timing(...)`. Đặt `useNativeDriver: true` để chạy trên UI thread.
- **Reanimated**: `useSharedValue` (giá trị dùng chung giữa JS và UI thread) + `useAnimatedStyle` (style tính trên UI thread) + các hàm `withTiming`/`withSpring`.

Angular có `@angular/animations` (trigger/state/transition). Ý tưởng tương tự, nhưng trên RN bạn làm việc với giá trị số và style trực tiếp.

Ví dụ

1. Animated API — hiện dần

`examples/src/chapters/ch04/FadeInView.tsx`:

```
export function FadeInView({ children, duration = 500 }: { children: ReactNode; duration?: number }) {
  // Tài liệu cũ hay viết useRef(new Animated.Value(0)).current – luật react-hooks/refs
  // (có trong eslint-config-expo 57) báo lỗi vì đọc ref khi render. useState(() => ...) tạo
  // giá trị một lần và an toàn.
  const [opacity] = useState(() => new Animated.Value(0));
  useEffect(() => {
    Animated.timing(opacity, { toValue: 1, duration, useNativeDriver: true }).start();
  }, [opacity, duration]);
  return <Animated.View testID="fade" style={{ opacity }}>{children}</Animated.View>;
}
```

Phát hiện khi viết sách: trang "Animations" của React Native vẫn dùng mẫu `useRef(new Animated.Value(0)).current`. Với `eslint-config-expo@57.0.2`, ESLint báo `Error: Cannot access refs during render (react-hooks/refs)`. Dùng `useState` với hàm khởi tạo.

2. Reanimated — nút tim này

`examples/src/components/FavoriteButton.tsx` (app mẫu):

```
const scale = useSharedValue(1);
const animatedStyle = useAnimatedStyle(() => ({ transform: [{ scale: scale.get() }] }));

const onPress = () => {
  // Dùng .set()/.get() thay vì gán .value — tương thích React Compiler (luật react-
  hooks/immutability).
  scale.set(withSequence(withSpring(1.4), withSpring(1)));
  toggle(postId);
  void tapFeedback();
};

<Animated.View style={animatedStyle}>
  <Text style={{ fontSize: 22 }}>{isFav ? '❤️' : '💔'}</Text>
</Animated.View>
```

Viết `scale.value = ...` vẫn chạy, nhưng ESLint (bộ luật React Compiler trong `eslint-config-expo`) báo `This value cannot be modified (react-hooks/immutability)`. Tài liệu `useSharedValue` của Reanimated khuyên: khi dùng React Compiler, dùng `get()/set()` thay vì `.value`.

3. Layout animation — thêm/xóa có hiệu ứng

```
{items.map((n) => (
  <Animated.View key={n} entering={FadeIn} exiting={FadeOut} layout={LinearTransition}>
    <Pressable accessibilityRole="button" onPress={() => setItems(l => l.filter(x => x !== n)}>
      <Text>Mục {n} (bấm để xóa)</Text>
    </Pressable>
  </Animated.View>
))})
```

App mẫu dùng `entering={FadeInDown.delay(Math.min(index, 10) * 40)}` trong `PostCard` để các thẻ trượt lên lần lượt.

Test

Cấu hình Jest cho Reanimated/Worklets (theo hướng dẫn "Testing with Jest" của Worklets và Reanimated):

```
"jest": {
  "preset": "jest-expo",
  "resolver": "react-native-worklets/jest/resolver",
  "setupFilesAfterEnv": [".jest.setup.js"]
}
```

```
// jest.setup.js
require('react-native-reanimated').setUpTests();
```

Không có `resolver`, test báo lỗi `TypeError: Cannot read properties of undefined (reading 'loadUnpackers')` (chúng tôi đã gặp). Lưu ý: tài liệu Reanimated nhắc `react-native-reanimated/jest/resolver`, nhưng gói `react-native-reanimated@4.5.1` đã cài không có thư mục `jest/; resolver` của `react-native-worklets@0.10.1` thì có.

Test "shake" dùng matcher `toHaveAnimatedStyle` của Reanimated và fake timers:

```
jest.useFakeTimers();
const view = await render(<ShakeOnError error={null} />);
expect(screen.getByTestId('shake')).toHaveAnimatedStyle({ transform: [{ translateX: 0 }] });
await view.rerender(<ShakeOnError error="Sai mật khẩu" />);
await act(async () => { jest.advanceTimersByTime(50); });
expect(screen.getByTestId('shake')).toHaveAnimatedStyle({ transform: [{ translateX: -10 }] });
```

Với Animated API + native driver, khung hình chạy trên UI thread nên Jest không thấy opacity đổi dần; ta kiểm tra giá trị đầu và cấu hình gửi cho `Animated.timing`.

Kết quả (2026-09-28):

```
PASS src/chapters/ch04/animation.test.tsx
  ✓ FadeInView: bắt đầu opacity 0 và chạy Animated.timing tới 1 bằng native driver
  ✓ bài tập: ShakeOnError lệch sang trái rồi về 0
  ✓ layout animation: thêm và xóa phần tử
Tests:      3 passed, 3 total
```

Trên Expo Go: tab **Lab** → "Ch.4 — Animation" (**NOT RUN**).

Đi sâu

Animated API hay Reanimated?

	Animated (có sẵn)	Reanimated 4
Cài thêm	Không	Có trong Expo Go SDK 57 (<code>react-native-reanimated@4.5.1</code> + <code>react-native-worklets@0.10.1</code>)
Chạy trên UI thread	Có, với <code>useNativeDriver: true</code> (một số thuộc tính)	Có, mặc định (worklet)
Gesture, layout animation	Hạn chế	Mạnh (<code>entering</code> , <code>exiting</code> , <code>layout</code>)
Khi nào dùng	Fade/slide đơn giản	Tương tác phức tạp, danh sách, gesture

RN 0.85 giới thiệu "Shared Animation Backend" viết cùng Software Mansion, dùng chung cho cả Animated và Reanimated (thử nghiệm, bật qua kênh experimental). Sách không bật tính năng này.

Worklet là gì?

Hàm trong `useAnimatedStyle(() => ...)` là **worklet**: được copy sang UI thread và chạy ở đó. Vì vậy trong worklet chỉ nên đọc shared value và tính toán nhẹ; không gọi `setState` trực tiếp.

Accessibility: giảm chuyển động

Người dùng có thể bật "Reduce Motion" trên iPhone. Reanimated có hook `useReducedMotion()`; RN CÓ `AccessibilityInfo.isReduceMotionEnabled()`. Với animation trang trí, hãy tắt hoặc rút ngắn khi người dùng bật tùy chọn này.

Lỗi và bẫy thường gặp

- **Quên** `useNativeDriver: true` → animation chạy trên JS thread, giật khi JS bận.
- `useNativeDriver: true` **VỚI** `width/height/top...`: theo mục "Caveats" của trang Animations, native driver chỉ animate thuộc tính **không phải layout** (`transform`, `opacity`). Blog RN 0.85 nói giới hạn này được gỡ khi bật Shared Animation Backend (kênh experimental) — sách không bật.
- **Gán** `sv.value` **trong component** → ESLint (React Compiler) báo lỗi; dùng `set()`.
- `useRef(new Animated.Value(0)).current` → ESLint `react-hooks/refs` báo lỗi; dùng `useState(() => ...)`.
- **Thiếu resolver trong Jest** → lỗi `loadUnpackers`.

Tóm tắt

- Animated API cho hiệu ứng đơn giản; Reanimated cho mọi thứ còn lại.
- Shared value + animated style + `withTiming/withSpring/withSequence`; layout animation bằng `entering/exiting/layout`.
- Viết theo chuẩn React Compiler: `get()/set()`, không đọc ref khi render.

Bài tập (có lời giải)

Bài 1. Viết `ShakeOnError({ error })`: mỗi khi `error` đổi (khác null), dòng lỗi rung ngang `-10` → `10` → `-6` → `0` trong 200 ms. Viết test bằng `toHaveAnimatedStyle`.

▼ Lời giải

`examples/src/chapters/ch04/ShakeOnError.tsx`:

```
export function ShakeOnError({ error }: { error: string | null }) {
  const x = useSharedValue(0);
  useEffect(() => {
    if (!error) return;
    x.set(
      withSequence(
        withTiming(-10, { duration: 50 }),
        withTiming(10, { duration: 50 }),
        withTiming(-6, { duration: 50 }),
        withTiming(0, { duration: 50 }),
      ),
    );
  }, [error, x]);
  const style = useAnimatedStyle(() => ({ transform: [{ translateX: x.get() }] }));
  return (
    <Animated.View testID="shake" style={style}>
      <Text style={{ color: '#dc2626' }}>{error ?? ' '}</Text>
    </Animated.View>
  );
}
```

Dùng kèm form đăng nhập: khi server trả "Sai mật khẩu", người dùng thấy ngay dòng lỗi rung.

Bài 2. Tôn trọng "Reduce Motion": nếu người dùng bật, `ShakeOnError` không rung. Sửa thể nào?

▼ Lời giải

```
import { useReducedMotion } from 'react-native-reanimated';

const reduceMotion = useReducedMotion();
useEffect(() => {
  if (!error || reduceMotion) return;
  x.set(withSequence(/* ... */));
}, [error, reduceMotion, x]);
```

Dòng lỗi vẫn hiện (thông tin không mất), chỉ bỏ hiệu ứng. (Lời giải tham khảo; chưa có test riêng trong dự án.)

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- React Native — Animations: <https://github.com/facebook/react-native-website/blob/main/docs/animations.md>
- React Native 0.85 blog (Shared Animation Backend): <https://github.com/facebook/react-native-website/blob/main/website/blog/2026-04-07-react-native-0.85.mdx>
- Reanimated — useSharedValue (React Compiler support): <https://github.com/software-mansion/react-native-reanimated/blob/main/docs/docs-reanimated/docs/core/useSharedValue.mdx>
- Reanimated — Testing with Jest: <https://github.com/software-mansion/react-native-reanimated/blob/main/docs/docs-reanimated/docs/guides/testing-with-jest.mdx>
- Worklets — Testing with Jest: <https://github.com/software-mansion/react-native-reanimated/blob/main/docs/docs-worklets/docs/guides/testing-with-jest.mdx>
- Reanimated — Entering/Exiting animations: <https://github.com/software-mansion/react-native-reanimated/blob/main/docs/docs-reanimated/docs/layout-animations/entering-exiting-animations.mdx>
- Expo — Reanimated (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/reanimated.mdx>

Chương 5 — Device APIs: rung, vị trí, chọn ảnh, quyền truy cập

Mục tiêu

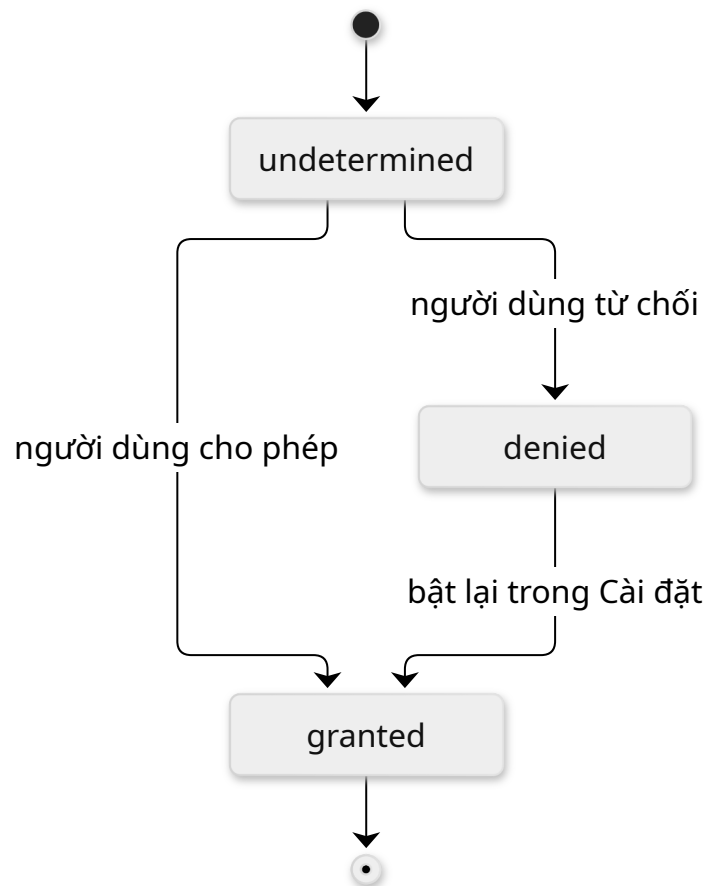
- Dùng 4 module Expo chạy được trong Expo Go SDK 57: `expo-haptics`, `expo-location`, `expo-image-picker`, `expo-network`.
- Xử lý **quyền truy cập (permission)** đúng cách: xin lần đầu, bị từ chối, bị từ chối vĩnh viễn.
- Tách logic thuần khỏi API native để test được.
- Mock module native trong Jest.

Giải thích đơn giản

Web có `navigator.geolocation`, `<input type="file">`, `navigator.vibrate`. Trên mobile, mỗi tính năng phần cứng là một **module native**. Expo đóng gói sẵn nhiều module; Expo Go có sẵn phần native của chúng, nên bạn chỉ cần `npx expo install <tên-gói>` và `import`.

Quy tắc quyền trên iOS:

- Lần đầu: app hỏi, iOS hiện hộp thoại hệ thống.
- Nếu người dùng từ chối: **không hỏi lại được** bằng code (`canAskAgain = false`). Chỉ còn cách mở **Cài đặt** (`Linking.openSettings()`).
- Khi build app thật, cần câu giải thích (ví dụ `NSLocationWhenInUseUsageDescription`) — Expo thêm qua config plugin trong `app.json`. Trong Expo Go, câu giải thích là của Expo Go.



Ví dụ

Logic quyền là hàm thuần

examples/src/chapters/ch05/permissions.ts:

```
export type PermissionStatus = 'granted' | 'denied' | 'undetermined';
export type PermissionAction = 'use' | 'ask' | 'open-settings';

export function nextPermissionAction(status: PermissionStatus, canAskAgain: boolean):
PermissionAction {
  if (status === 'granted') return 'use';
  if (status === 'undetermined' || canAskAgain) return 'ask';
  return 'open-settings'; // iOS: đã từ chối → chỉ bật lại được trong Cài đặt
}
```

Vị trí (expo-location)

examples/src/chapters/ch05/LocationCard.tsx (trích):

```
const locate = async () => {
  setState({ kind: 'loading' });
  try {
    let perm = await Location.getForegroundPermissionsAsync();
    let action = nextPermissionAction(perm.status as PermissionStatus, perm.canAskAgain);
    if (action === 'ask') {
      perm = await Location.requestForegroundPermissionsAsync();
      action = nextPermissionAction(perm.status as PermissionStatus, perm.canAskAgain);
    }
    if (action !== 'use') {
      setState({ kind: 'blocked' });
      return;
    }
    const pos = await Location.getCurrentPositionAsync({ accuracy: Location.Accuracy.Balanced });
    setState({ kind: 'done', text: formatCoords(pos.coords.latitude, pos.coords.longitude) });
  } catch (e) {
    setState({ kind: 'error', message: e instanceof Error ? e.message : String(e) });
  }
};
```

State dạng **discriminated union** (`kind: 'idle' | 'loading' | 'done' | 'blocked' | 'error'`) giúp không bao giờ có trạng thái vô lý như "đang tải và có lỗi cùng lúc".

Chọn ảnh (expo-image-picker)

```
const result = await ImagePicker.launchImageLibraryAsync({
  mediaTypes: ['images'],
  allowsEditing: true,
  aspect: [1, 1],
  quality: 0.7,
});
if (!result.canceled) {
  setUri(result.assets[0].uri);
  void successFeedback();
}
```

Tài liệu expo-image-picker SDK 57 ghi trong ví dụ: "No permissions request is necessary for launching the image library" (chỉ trường hợp video đặc biệt trên iOS mới nên xin trước).

Rung (expo-haptics)

`examples/src/lib/haptics.ts`:

```
export async function tapFeedback(): Promise<void> {
  if (Platform.OS === 'web') return;
  try {
    await Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
  } catch {
    // Một số thiết bị không hỗ trợ rung – không sao.
  }
}
```

Nút tìm trong app mẫu gọi `tapFeedback()`; chọn ảnh xong gọi `successFeedback()` (`Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success)`).

Mạng (expo-network)

`OfflineBanner` dùng hook `useNetworkState()`; `src/lib/queryClient.ts` dùng `addNetworkStateListener` + `getNetworkStateAsync` cho TanStack Query (Chương 2).

Test: mock module native

```
jest.mock('expo-location', () => ({
  Accuracy: { Balanced: 3 },
  getForegroundPermissionsAsync: jest.fn(),
  requestForegroundPermissionsAsync: jest.fn(),
  getCurrentPositionAsync: jest.fn(),
}));

const loc = jest.mocked(Location);
beforeEach(() => jest.clearAllMocks()); // xóa số lần gọi giữa các test

it('LocationCard: đã từ chối vĩnh viễn → gợi ý mở Cài đặt', async () => {
  loc.getForegroundPermissionsAsync.mockResolvedValue({ status: 'denied', canAskAgain:
false } as never);
  const user = userEvent.setup();
  await render(<LocationCard />);
  await user.press(screen.getByRole('button', { name: '📍 Lấy vị trí của tôi' }));
  expect(await screen.findByText('Bạn đã tắt quyền vị trí. Mở Cài
đặt')).toBeOnTheScreen();
  expect(loc.getCurrentPositionAsync).not.toHaveBeenCalled();
});
```

Kết quả (2026-09-28):

```
PASS src/chapters/ch05/device.test.tsx
  ✓ nextPermissionAction
  ✓ bài tập: formatCoords
  ✓ LocationCard: xin quyền lần đầu rồi hiện tọa độ
  ✓ LocationCard: đã từ chối vĩnh viễn → gợi ý mở Cài đặt
  ✓ AvatarPicker: chọn ảnh xong thì hiện ảnh và rung
Tests:      5 passed, 5 total
```

Lần đầu viết, test "đã từ chối vĩnh viễn" **thất bại** ("Expected number of calls: 0, Received: 1") vì số lần gọi mock còn sót từ test trước. Thêm `beforeEach(() => jest.clearAllMocks())` là hết.

Trên Expo Go: tab **Thiết bị** (NOT RUN — cần iPhone thật để thấy hộp thoại quyền và rung).

Đi sâu

Giới hạn của Expo Go (theo tài liệu SDK 57)

- Vị trí **nền** (background location) trên iOS: **không** có trong Expo Go → cần development build.
- Push notification từ xa: không có trong Expo Go trên Android từ SDK 53 (thông báo cục bộ vẫn được).
- Face ID của `expo-local-authentication`: không có trong Expo Go trên iOS.

Chọn độ chính xác vị trí

`Accuracy.Balanced` đủ cho "thành phố/quận", nhanh và tiết kiệm pin hơn `High/Highest`. Chỉ xin quyền **khi người dùng bấm** nút cần vị trí (không xin ngay lúc mở app): lúc đó người dùng hiểu vì sao app cần quyền, nên ít khi từ chối vĩnh viễn.

Thiết kế cho test

Ba lớp:

1. Hàm thuần (`nextPermissionAction`, `formatCoords`) — test không cần mock.
2. Wrapper mỏng quanh module native (`lib/haptics.ts`) — nuốt lỗi, bỏ qua web.
3. Component — test với `jest.mock` của module native.

Giống Angular: bọc API trình duyệt trong service rồi mock service trong TestBed.

Lỗi và bẫy thường gặp

- **Xin quyền lúc khởi động app** → người dùng dễ từ chối vĩnh viễn.
- **Không xử lý** `canAskAgain = false` → nút bấm "không làm gì", người dùng bối rối.
- **Quên** `result.canceled` khi người dùng đóng trình chọn ảnh → `assets` là `null`.
- **Gọi haptics trên web** → không hỗ trợ; bọc wrapper.
- **Mock không reset giữa các test** → kết quả sai lệch.
- **Thêm thư viện native không có trong Expo Go** → màn hình đỏ "native module not found"; cần development build (Tập 3).

Tóm tắt

- Expo Go SDK 57 có sẵn haptics, location, image picker, network.
- Luồng quyền: undetermined → ask; denied + canAskAgain=false → mở Cài đặt.
- Tách hàm thuần + wrapper; mock module native trong test.

Bài tập (có lời giải)

Bài 1. Viết `formatCoords(lat, lng)` hiển thị kiểu Việt Nam, dấu phẩy thập phân, 4 chữ số: `10.77689, 106.70092` → `10,7769° B, 106,7009° Đ`; số âm dùng N (Nam) và T (Tây).

▼ Lời giải

`examples/src/chapters/ch05/formatCoords.ts`:

```
export function formatCoords(latitude: number, longitude: number, digits = 4):
string {
  const fmt = (n: number) => Math.abs(n).toFixed(digits).replace('.', ',');
  const ns = latitude >= 0 ? 'B' : 'N'; // Bắc / Nam
  const ew = longitude >= 0 ? 'Đ' : 'T'; // Đông / Tây
  return `${fmt(latitude)}° ${ns}, ${fmt(longitude)}° ${ew}`;
}
```

Test: `formatCoords(-33.8688, -70.5) === '33,8688° N, 70,5000° T'`.

Bài 2. Thêm nút "Chụp ảnh" dùng camera thay vì thư viện. Cần đổi gì?

▼ Lời giải

Dùng `ImagePicker.launchCameraAsync(...)` với cùng tùy chọn. Camera **cần quyền**:

```
const perm = await ImagePicker.requestCameraPermissionsAsync();
if (!perm.granted) { /* dùng nextPermissionAction(perm.status, perm.canAskAgain)
để quyết định */ return; }
const result = await ImagePicker.launchCameraAsync({ allowsEditing: true, aspect:
[1, 1], quality: 0.7 });
```

Test: mock thêm `requestCameraPermissionsAsync` và `launchCameraAsync` giống test `AvatarPicker`. Simulator iOS không có camera, nên phải thử trên máy thật. (Lời giải tham khảo, chưa có trong code dự án.)

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Expo — Haptics (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/haptics.mdx>
- Expo — Location (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/location.mdx>
- Expo — ImagePicker (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/imagepicker.mdx>
- Expo — Network (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/network.mdx>
- Expo — Notifications (giới hạn Expo Go): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/notifications.mdx>
- Expo — LocalAuthentication (giới hạn Expo Go): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/local-authentication.mdx>
- React Native — Linking (openSettings): <https://github.com/facebook/react-native-website/blob/main/docs/linking.md>

Chương 6 — Testing: jest-expo và React Native Testing Library

Mục tiêu

- Hiểu bộ công cụ test của sách: **Jest 29 + jest-expo 57 + React Native Testing Library (RNTL) 14**.
- Viết test theo "kim tự tháp": hàm thuần → component → toàn app.
- Mock đúng cách: `fetch`, `module native`, `Alert`, thời gian (fake timers).
- Đọc báo cáo **coverage** (độ phủ).

Giải thích đơn giản

- **Jest**: trình chạy test (giống Karma/Jasmine hoặc Jest trong Angular).
- **jest-expo**: "preset" cấu hình sẵn Jest cho Expo: biên dịch TypeScript/JSX bằng Babel, giả lập môi trường React Native (mặc định iOS), mock nhiều module Expo.
- **RNTL**: render component trong bộ nhớ (không cần điện thoại) và cho bạn tìm phần tử **như người dùng thấy**: theo vai trò (`getByRole`), nhãn (`getByLabelText`), chữ (`getByText`).

Quy tắc vàng của RNTL: "The more your tests resemble the way your software is used, the more confidence they can give you." — test càng giống cách người dùng dùng app, càng đáng tin.

Ví dụ

1. Tầng hàm thuần

Không cần render, chạy cực nhanh. Ví dụ `loadPosts` (Chương 3) với cache trong bộ nhớ:

```
it('mất mạng: dùng cache', async () => {
  const cache = memoryCache({ posts: [POSTS[0]], savedAt: 1 });
  const r = await loadPosts({ fetcher: async () => Promise.reject(new
    ApiError('offline', null)), cache });
  expect(r.source).toBe('cache');
});
```

2. Tầng component — mock `Alert`

`Alert.alert` là API native: trong test nó không hiện hộp thoại. Ta **spy** (theo dõi) nó, rồi tự "bấm" nút trong hộp thoại (`examples/src/chapters/ch06/testing.test.tsx`):

```
function pressAlertButton(spy: jest.SpyInstance, text: string) {
  const buttons = spy.mock.calls[0][2] as AlertButton[];
  buttons.find((b) => b.text === text)?.onPress?.();
}

it('Alert: bấm "Xóa" thì gọi onConfirm', async () => {
  const alertSpy = jest.spyOn(Alert, 'alert').mockImplementation(() => {});
  const onConfirm = jest.fn();
  const user = userEvent.setup();
  await render(<ConfirmDeleteButton itemName="Bài 1" onConfirm={onConfirm} />);

  await user.press(screen.getByRole('button', { name: 'Xóa Bài 1' }));
  expect(alertSpy).toHaveBeenCalledWith('Xóa?', 'Bài 1', expect.any(Array));

  pressAlertButton(alertSpy, 'Xóa');
  expect(onConfirm).toHaveBeenCalledTimes(1);
  alertSpy.mockRestore();
});
```

Đây là phần còn thiếu ở app Tập 1 (luồng "Xóa việc" dùng `Alert`).

3. Tăng toàn app — `renderRouter` + mạng giả

`examples/src/__tests__/app.test.tsx`:

```
jest.mock('expo-network', () => ({
  useNetworkState: () => ({ isConnected: true, isInternetReachable: true }),
  addNetworkStateListener: () => ({ remove: () => {} }),
  getNetworkStateAsync: async () => ({ isConnected: true }),
}));

it('mất mạng: hiện dữ liệu đã lưu trong cache', async () => {
  await asyncStoragePostsCache.save([POSTS[2]], new Date(2026, 8, 28, 8, 5).getTime());
  fetchMock.mockRejectedValue(new TypeError('Network request failed'));
  await renderApp();
  expect(await screen.findByText('Dữ liệu offline, lưu lúc 08:05 28/09')).toBeOnTheScreen();
  expect(screen.getByText('TanStack Query cho người mới')).toBeOnTheScreen();
});
```

Test này đi qua **mọi lớp thật**: layout gốc, `QueryClient`, `usePosts`, `loadPosts`, `AsyncStorage` (bản giả của thư viện), màn hình. Chỉ `fetch` và `expo-network` là giả.

Kết quả toàn bộ Tập 2 (2026-09-28)

```
PASS src/__tests__/app.test.tsx
```

```
App Tập 2: Tin Đọc Sau
```

- ✓ tải bài viết từ mạng, mở chi tiết
- ✓ mất mạng: hiện dữ liệu đã lưu trong cache
- ✓ mất mạng và chưa có cache: báo lỗi + nút Thử lại
- ✓ yêu thích một bài → xuất hiện ở tab Yêu thích

```
Tab Lab
```

- ✓ mở ví dụ Zustand

```
PASS src/chapters/ch06/testing.test.tsx
```

```
✓ Alert: bấm "Xóa" thì gọi onConfirm
✓ Alert: bấm "Hủy" thì không xóa
✓ bài tập: OfflineBanner với { isConnected: false, isInternetReachable: false } → hiện =
true
✓ bài tập: OfflineBanner với { isConnected: true, isInternetReachable: false } → hiện =
true
✓ bài tập: OfflineBanner với { isConnected: true, isInternetReachable: true } → hiện =
false
✓ bài tập: OfflineBanner với { isConnected: true, isInternetReachable: undefined } → hiện
= false
...
Test Suites: 11 passed, 11 total
Tests:      43 passed, 43 total
```

Coverage (npx jest --coverage --coverageReporters=text-summary):

```
===== Coverage summary =====
Statements   : 92.09% ( 338/367 )
Branches     : 81.56% ( 115/141 )
Functions    : 88.95% ( 153/172 )
Lines        : 92.69% ( 279/301 )
=====
```

Đi sâu

Cấu hình Jest của Tập 2

```
"jest": {
  "preset": "jest-expo",
  "resolver": "react-native-worklets/jest/resolver",
  "setupFilesAfterEnv": [".jest.setup.js"]
}
```

`jest.setup.js` bật matcher của Reanimated và mock AsyncStorage (xem Chương 3, 4).

Chọn query (theo hướng dẫn RNTL v14)

Thứ tự ưu tiên: `getByRole` → `getByLabelText` → `getByPlaceholderText` → `getByText` → `getByDisplayValue` → `getByTestId` (cuối cùng).

- `getBy*`: phải có, không có thì lỗi.
- `queryBy*`: chỉ dùng để khẳng định **không có** (`.not.toBeOnTheScreen()`).
- `findBy*`: chờ phần tử xuất hiện (sau API, timer).

RNTL 14: mọi thứ là async

`await render(...)`, `await fireEvent(...)`, `await renderHook(...)`, `await user.press(...)`, `await view.render(...)`, `await view.unmount()`. Quên `await` → cảnh báo `act(...)` hoặc test chạy sai thứ tự.

Ba cách mock

Mock gì	Cách	Ví dụ trong sách
Module cả gói	<code>jest.mock('expo-location', () => ({ ... }))</code>	ch05 <code>device.test.tsx</code>
Một hàm của object	<code>jest.spyOn(Alert, 'alert')</code>	ch06 <code>testing.test.tsx</code>
Global	<code>globalThis.fetch = jest.fn()</code>	ch02, app test
Dependency qua tham số	truyền <code>fetchImpl</code> , <code>cache</code> , <code>now</code>	<code>createApiClient</code> , <code>loadPosts</code>

Cách cuối (inject qua tham số) là dễ nhất và ít "phép màu" nhất — giống DI của Angular.

So với Angular testing

Angular	Sách này
<code>TestBed.configureTestingModule</code>	<code>renderWithProviders(ui)</code> tạo <code>QueryClient</code> mới
<code>HttpTestingController.expectOne(url).flush(data)</code>	<code>fetchMock.mockResolvedValueOnce(jsonResponse(c</code>
<code>fakeAsync / tick</code>	<code>jest.useFakeTimers()</code> / <code>jest.advanceTimersByTime</code>
<code>By.css</code> , <code>nativeElement.textContent</code>	<code>getByRole</code> , <code>toHaveTextContent</code>
Cypress/Playwright E2E	Maestro/Detox (ngoài phạm vi sách; cần simulator)

Lỗi và bẫy thường gặp

- Quên `await` với API RNTL 14.
- Mock rò giữa các test → `jest.clearAllMocks()` / `mockReset()` trong `beforeEach`.
- Store/cache singleton (Zustand, AsyncStorage giả) giữ dữ liệu giữa các test → reset trong `beforeEach`.
- `QueryClient` dùng chung giữa các test → tạo mới mỗi test, `retry: false`.
- Tìm theo `testID` khắp nơi → test không phản ánh người dùng; ưu tiên `role/label/text`.
- `jest.spyOn(RN, 'useWindowDimensions')` không tác dụng (xem Tập 1, Chương 4) → tách component nhận giá trị qua props.
- Snapshot test lớn → dễ vỡ, khó review. Sách không dùng snapshot.

Tóm tắt

- Jest + jest-expo + RNTL 14; ưu tiên test hành vi qua role/label/text.
- Kim tự tháp: nhiều test thuần, vừa đủ test component, vài test toàn app.
- Mock ít nhất có thể; inject dependency qua tham số khi được.

Bài tập (có lời giải)

Bài 1. Viết test cho `OfflineBanner` với 4 trạng thái mạng, dùng `it.each` và mock `expo-network`.

▼ Lời giải

```
jest.mock('expo-network', () => ({ useNetworkState: jest.fn() }));

it.each([
  [{ isConnected: false, isInternetReachable: false }, true],
  [{ isConnected: true, isInternetReachable: false }, true],
  [{ isConnected: true, isInternetReachable: true }, false],
  [{ isConnected: true, isInternetReachable: undefined }, false],
])('bài tập: OfflineBanner với %o → hiện = %p', async (state, visible) => {
  jest.mocked(useNetworkState).mockReturnValue(state as never);
  await render(<OfflineBanner />);
  const banner = screen.queryByRole('alert');
  if (visible) expect(banner).toBeOnTheScreen();
  else expect(banner).not.toBeOnTheScreen();
});
```

Trường hợp `isInternetReachable: undefined` (chưa biết) quan trọng: banner **không** được hiện, nếu không app sẽ nhấp nháy "offline" mỗi lần mở.

Bài 2. Viết test cho luồng "Xóa việc" của app Tập 1 (`src/app/task/[id].tsx`), dùng kỹ thuật `Alert` ở trên.

▼ Lời giải

Trong `vol1-co-ban/examples/src/__tests__/app.routes.test.tsx`:

```
it('xóa việc sau khi xác nhận', async () => {
  const alertSpy = jest.spyOn(Alert, 'alert').mockImplementation(() => {});
  const app = await renderApp('/task/seed-2');
  await userEvent.setup().press(await screen.findByRole('button', { name: 'Xóa' }));
  const buttons = alertSpy.mock.calls[0][2] as AlertButton[];
  await act(async () => { buttons.find((b) => b.text === 'Xóa')?.onPress?.(); });
  expect(app.getPathname()).toBe('/');
  expect(screen.queryByText('Đọc chương Flexbox')).not.toBeOnTheScreen();
  alertSpy.mockRestore();
});
```

Test này đã được thêm vào Tập 1 và **tìm ra một lỗi thật**: khi mở thẳng `/task/seed-2` rồi xóa, `router.back()` không có chỗ quay lại ("The action 'GO_BACK' was not handled by any navigator"). Tập 1 đã sửa bằng hàm `goBackOr(router, '/')` (xem Tập 1, Chương 7). Đây là lý do nên có test tích hợp.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- React Native Testing Library — repo và README: <https://github.com/callstack/react-native-testing-library>
- RNTL — LLM guidelines v14 (query, async): <https://github.com/callstack/react-native-testing-library/blob/main/docs/guides/llm-guidelines.md>
- RNTL — Migration to v14: <https://github.com/callstack/react-native-testing-library/blob/main/docs/guides/migration-v14.md>
- Expo — Unit testing with Jest (mã nguồn docs): <https://github.com/expo/expo/blob/main/docs/pages/develop/unit-testing.mdx>
- React Native — Testing overview: <https://github.com/facebook/react-native-website/blob/main/docs/testing-overview.md>
- TanStack Query — Testing: <https://github.com/TanStack/query/blob/main/docs/framework/react/guides/testing.md>

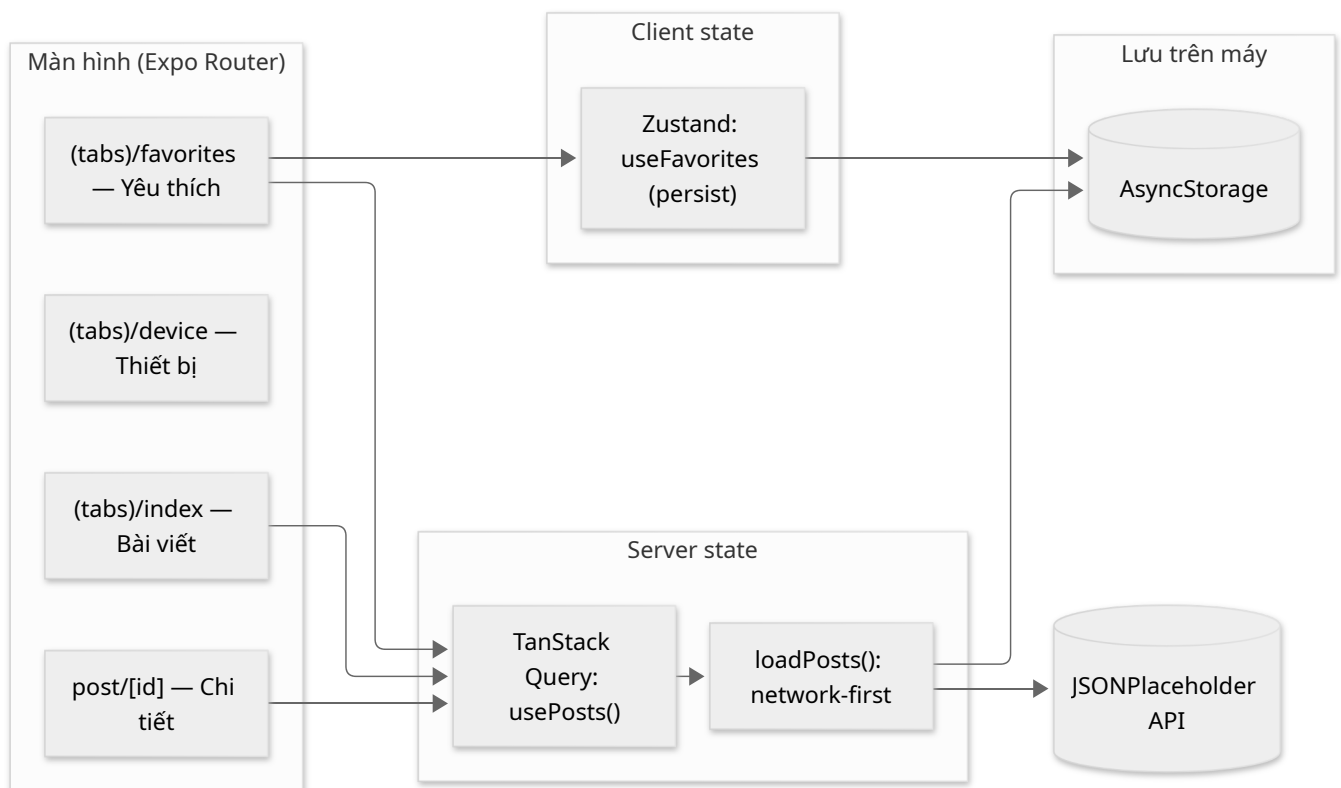
Chương 7 — App mẫu Tập 2: "Tin Đọc Sau"

Mục tiêu

Ghép Tập 2 thành một app chạy trong Expo Go:

- Tab **Bài viết**: tải từ API (TanStack Query), kéo để làm mới, **offline** vẫn xem được (cache AsyncStorage).
- Tab **Yêu thích**: store Zustand có persist; badge số lượng trên tab.
- Tab **Thiết bị**: chọn ảnh đại diện, lấy vị trí, rung.
- Tab **Lab**: ví dụ từng chương.
- Animation: thẻ bài trượt vào, tìm nãy; rung khi bấm.

Giải thích đơn giản



- **Server state** (bài viết) chỉ nằm trong TanStack Query; tab Yêu thích **dùng lại cache** đó.
- **Client state** (id yêu thích) nằm trong Zustand.
- Hai thứ gặp nhau ở màn hình: `posts.filter((p) => ids.includes(p.id))`.

Ví dụ

Chạy app (lệnh chính xác)

```
cd books/react-native/vol2-trung-cap/examples
npm ci
npx expo login          # cùng tài khoản với Expo Go trên iPhone
npx expo start          # quét QR bằng Camera
```

Thử offline: mở app khi có mạng (để có cache) → bật Chế độ máy bay → kéo để làm mới hoặc mở lại app → thấy "Đang offline — hiển thị dữ liệu đã lưu" và "Dữ liệu offline, lưu lúc ...".

Kiểm tra chất lượng:

```
npm run typecheck && npm run lint && npm test
npm run export:ios    # tùy chọn: bundle Hermes cho iOS
```

Màn hình Bài viết

examples/src/app/(tabs)/index.tsx (trích):

```
const { data, isPending, isError, error, refetch, isRefetching } = usePosts();

if (isPending) return <View style={styles.center}><ActivityIndicator accessibilityLabel="Đang tải" /></View>;
if (isError)
  return (
    <View style={styles.center}>
      <Text style={styles.error}>Không tải được bài viết: {error.message}</Text>
      <AppButton title="Thử lại" onPress={() => refetch()} />
    </View>
  );

return (
  <View style={styles.screen}>
    <OfflineBanner />
    {data.source === 'cache' ? <Text style={styles.note}>Dữ liệu offline, lưu lúc {formatSavedAt(data.savedAt)}</Text> : null}
    <FlatList
      data={data.posts}
      keyExtractor={(p) => String(p.id)}
      renderItem={({ item, index }) => <PostCard post={item} index={index} onOpen={open} />}
      refreshing={isRefetching}
      onRefresh={() => refetch()}
    />
  </View>
);
```

Badge trên tab

```
const favCount = useFavoriteCount(); // selector: s.ids.length
<Tabs.Screen name="favorites" options={{ title: 'Yêu thích', tabBarBadge: favCount > 0 ?
favCount : undefined }} />
```

Layout Tabs chỉ render lại khi **số lượng** đổi, không phải mỗi khi store đổi.

Kết quả kiểm tra toàn bộ Tập 2 (chạy thật 2026-09-28)

```
bash books/react-native/scripts/check-all.sh vol2-trung-cap
```

```
--- typecheck
--- lint
--- test
Test Suites: 11 passed, 11 total
Tests:       43 passed, 43 total
--- export web
web bundle OK
--- export ios
ios bundle OK
ALL CHECKS PASSED: vol2-trung-cap
```

Chạy trên Expo Go: NOT RUN (không có iPhone trong sandbox). Sandbox cũng **không truy cập được** `jsonplaceholder.typicode.com` (bị proxy chặn), nên mọi test dùng `fetch` giả; cấu trúc dữ liệu được lấy từ `data.json` trong repo `typicode/jsonplaceholder`.

Đi sâu

Vì sao không để bài viết trong Zustand?

Nếu copy danh sách bài vào Zustand, bạn có **hai nguồn sự thật**: cache của TanStack Query và store. Chúng sẽ lệch nhau khi refetch. Quy tắc: dữ liệu của server → TanStack Query; lựa chọn của người dùng → Zustand; chỉ lưu **id** để nối.

Cấu trúc thư mục

```
examples/src/
  api/           # client fetch, hàm gọi API, query keys (+ test)
  storage/       # PostsCache: AsyncStorage và SQLite (+ test với node:sqlite)
  state/         # Zustand stores (+ test)
  features/posts/ # loadPosts (network-first), usePosts, format (+ test)
  components/    # PostCard, FavoriteButton, OfflineBanner, AppButton
  lib/           # queryClient (onlineManager/focusManager), haptics
  app/           # màn hình Expo Router
  chapters/chNN/ # ví dụ + lời giải từng chương (+ test)
  test/          # fixtures, renderWithProviders, adapter node:sqlite
  __tests__/     # test toàn app
```

Hạn chế có chủ ý (Tập 3 sẽ xử lý)

- JSONPlaceholder là API giả: tạo bài mới không được lưu thật.
- Chưa có đăng nhập / token → Tập 3 (Security, expo-secure-store).
- Danh sách 20 bài nên FlatList đủ nhanh → Tập 3 so sánh FlashList.
- Chưa có báo lỗi từ xa (crash reporting) → Tập 3 (Monitoring).

Lỗi và bẫy thường gặp

- **Tạo QueryClient trong component không có useState** → mất cache mỗi lần render.
- **Không nối onlineManager** → không tự refetch khi có mạng lại.
- **Coi isInternetReachable: undefined là offline** → banner nhấp nháy khi mở app.
- **Dùng index làm key** trong danh sách có thể thay đổi → animation entering chạy sai phần tử.
- **Quên expo-asset** khi dùng expo-sqlite → Jest không tìm thấy module (Chương 3).

Tóm tắt

- Server state (TanStack Query) + client state (Zustand) + lưu trữ (AsyncStorage/SQLite) tách bạch.
- Offline-first bằng "network-first, cache fallback" + banner trạng thái mạng.
- 43 test, gồm test toàn app mô phỏng có mạng / mất mạng.

Bài tập (có lời giải)

Bài 1. Ở tab Yêu thích, nếu một bài yêu thích **không còn** trong danh sách tải về (API đổi), hãy hiện dòng "N bài không còn trên máy chủ".

▼ Lời giải

```
const available = new Set((data?.posts ?? []).map((p) => p.id));
const missing = ids.filter((id) => !available.has(id)).length;
{missing > 0 ? <Text>{missing} bài không còn trên máy chủ</Text> : null}
```

Test: `useFavorites.getState().toggle(999)` trước khi render, `fetch` trả `POSTS` (không có id 999), mở tab Yêu thích, mong đợi "1 bài không còn trên máy chủ". (Lời giải tham khảo; chưa có trong code dự án.)

Bài 2. Thêm nút "Xóa cache" trong tab Thiết bị để thử lại kịch bản "mất mạng và chưa có cache".

▼ Lời giải

```
import { useQueryClient } from '@tanstack/react-query';
import { asyncStoragePostsCache } from '@storage/postsCache';

const qc = useQueryClient();
<AppButton
  title="Xóa cache bài viết"
  variant="ghost"
  onPress={async () => {
    await asyncStoragePostsCache.clear(); // xóa bản lưu trên máy
    qc.removeQueries({ queryKey: postKeys.all }); // xóa cache trong bộ nhớ
  }}
/>
```

Phải xóa **cả hai** tầng cache: AsyncStorage và TanStack Query. (Lời giải tham khảo.)

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- TanStack Query — React Native: <https://github.com/TanStack/query/blob/main/docs/framework/react/react-native.md>
- Zustand — Persisting store data: <https://github.com/pmndrs/zustand/blob/main/docs/reference/integrations/persisting-store-data.md>
- Expo Router — Navigation layouts (Tabs): <https://github.com/expo/expo/blob/main/docs/pages/router/basics/navigation-layouts.mdx>
- Expo — Network (SDK 57): <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/network.mdx>
- JSONPlaceholder: <https://github.com/typicode/jsonplaceholder>

GLOSSARY — Thuật ngữ React Native (Anh → Việt)

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular
React Native (RN)	Framework viết app iOS/Android bằng React; vẽ view native thật	Ionic/Capacitor (nhưng Ionic dùng WebView)
Expo	Framework cho RN: CLI, router; thư viện module native, dịch vụ build	Angular CLI + thư viện chính thức
Expo SDK	Bộ phiên bản Expo đi cùng một phiên bản RN (sách dùng SDK 57 ↔ RN 0.86)	Phiên bản Angular major
Expo Go	App trên App Store để chạy thử project mà không cần build	—
Development build	Bản app "Expo Go của riêng bạn", có native module tùy ý	—
EAS (Expo Application Services)	Dịch vụ cloud: build, submit, update	CI/CD dịch vụ
Metro	Bundler (công cụ đóng gói JS) của RN	esbuild/webpack trong Angular CLI
Hermes	JavaScript engine chạy trên điện thoại; V1 mặc định từ RN 0.84	V8 trong trình duyệt
Fast Refresh	Cập nhật app ngay khi lưu file, giữ state	HMR (<code>ng serve --hmr</code>)
New Architecture	Kiến trúc mới của RN: JSI + Fabric + TurboModules; bắt buộc từ RN 0.82	—
JSI (JavaScript Interface)	Lớp C++ cho JS gọi native trực tiếp, không qua "bridge"	—
Fabric	Hệ thống render mới của RN	Renderer
TurboModules	Native module thế hệ mới, tải lười (lazy)	—
Codegen	Sinh code C++/native từ type TypeScript	—
Component	Hàm trả về JSX	<code>@Component</code>
JSX	Cú pháp giống HTML viết trong TypeScript	Template
Props	Tham số truyền vào component (chỉ đọc)	<code>@Input()</code>
Callback prop	Hàm truyền xuống để con báo sự kiện lên	<code>@Output()</code>
State	Dữ liệu thay đổi được của component (<code>useState</code>)	<code>signal()</code> / field của class
Hook	Hàm <code>use...</code> dùng tính năng React trong component hàm	Lifecycle hooks + <code>inject()</code>
Effect (<code>useEffect</code>)	Side effect sau khi render, có hàm dọn dẹp	<code>ngOnInit</code> + <code>ngOnDestroy</code> , <code>effect()</code>
Memo (<code>useMemo</code> , <code>memo</code>)	Ghi nhớ kết quả/Component để tránh tính/render lại	<code>computed()</code> , <code>OnPush</code>
Context	Truyền giá trị xuống cây component không qua props	Dependency Injection
Controlled component	Input mà giá trị nằm trong state React	<code>[(ngModel)]</code> , <code>FormControl</code>
Pressable	Component bắt sự kiện chạm	<code>(click)</code> trên <code><button></code>

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular
FlatList / SectionList	Danh sách ảo hóa (virtualized), chỉ render dòng đang thấy	<code>cdk-virtual-scroll</code>
Virtualization	Chỉ render phần tử đang nằm trên màn hình	Virtual scroll
Flexbox	Hệ thống bố cục; RN mặc định <code>flexDirection: 'column'</code>	CSS Flexbox
StyleSheet	API tạo style cho component	SCSS của component
Safe area	Vùng không bị tai thỏ/thanh home che	—
Expo Router	Router theo file (file-based routing) cho Expo	Angular Router
Layout (<code>_layout.tsx</code>)	File định nghĩa navigator (Stack, Tabs) cho một thư mục	Component chứa <code><router-outlet></code>
Stack / Tabs	Kiểu điều hướng: chồng màn hình / thanh tab dưới	—
Deep link	Link mở thẳng vào một màn hình trong app	URL route
Accessibility (a11y)	Khả năng tiếp cận: VoiceOver, vai trò, nhãn	ARIA
jest-expo	Preset Jest của Expo	Karma/Jest preset Angular
RNTL	React Native Testing Library — test theo cách người dùng dùng app	TestBed + Testing Library
Continuous Native Generation (CNG)	Thư mục <code>ios/</code> và <code>android/</code> được sinh ra từ <code>app.json</code> , không sửa tay	—
Config plugin	Plugin sửa cấu hình native khi sinh <code>ios/ / android/</code>	Angular schematics (gần giống)